Adam: A Method for Stochastic Optimization

Study, Implementation, and Empirical Evaluation

Group Members

Nishant Malhotra (2022EEB1196)
Shivam Zample (2022MCB1280)
Nishant Yadav (2022CSB1098)
Harsh Patidar (2022MCB1265)

1 Introduction

Gradient-based optimization is central to training modern machine learning models. Standard Stochastic Gradient Descent (SGD) uses a single, fixed learning rate for all parameters, making it sensitive to hyperparameter choice and often slow to converge on ill-conditioned problems.

Adam (Adaptive Moment Estimation), proposed by Kingma and Ba [**?**], addresses these shortcomings by maintaining per-parameter adaptive learning rates derived from running estimates of the first and second moments of the gradients. The method is computationally efficient, requires little memory, and is invariant to diagonal rescaling of the gradients, making it well-suited for large-scale deep learning tasks.

This report presents: (i) a from-scratch Python implementation of Adam, (ii) a comparison with PyTorch's built-in optimizer on MNIST, (iii) a discussion of the algorithm's theoretical convergence guarantees, and (iv) an ablation study over the key hyperparameters β_1 and β_2.

2 Algorithm

2.1 Update Rules

Adam maintains two exponential moving averages of the gradient at each timestep t:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2 \tag{2}$$

where $g_t = \nabla_\theta f_t(\theta_{t-1})$ is the stochastic gradient. Since $m_0 = v_0 = \mathbf{0}$, the estimates are biased toward zero during early training. Bias-corrected estimates are:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{3}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4}$$

The parameter update rule is then:

$$\theta_t = \theta_{t-1} - \alpha \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{5}$$

The recommended default hyperparameters are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$.

2.2 Pseudocode

Algorithm 1 Adam Optimizer [**?**]

Require: Stepsize α; decay rates $\beta_1, \beta_2 \in [0, 1)$; $\varepsilon > 0$; initial parameters θ_0

$m_0 \leftarrow \mathbf{0}$; $\quad v_0 \leftarrow \mathbf{0}$; $\quad t \leftarrow 0$
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ *// stochastic gradient*
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$ *// biased first moment*
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$ *// biased second moment*
  $\hat{m}_t \leftarrow m_t \,/\, (1 - \beta_1^t)$ *// bias-corrected first moment*
  $\hat{v}_t \leftarrow v_t \,/\, (1 - \beta_2^t)$ *// bias-corrected second moment*
  $\theta_t \leftarrow \theta_{t-1} - \alpha\, \hat{m}_t \,/\, (\sqrt{\hat{v}_t} + \varepsilon)$ *// parameter update*
end while
return θ_t

2.3 Key Properties

- **Adaptive learning rates.** Each parameter receives its own effective learning rate, scaled by the inverse square root of its second moment estimate.
- **Bias correction.** Dividing by $(1 - \beta^t)$ counteracts the downward bias introduced by zero-initialization of m_0 and v_0, which is especially important in the early timesteps and when β_2 is close to 1.
- **Scale invariance.** Rescaling the gradient by a constant c scales $\hat{m}_t$ by c and $\hat{v}_t$ by c^2, which cancel in the update ratio, leaving the step direction unchanged.
- **Automatic annealing.** Near a local optimum the signal-to-noise ratio $\hat{m}_t/\sqrt{\hat{v}_t} \to 0$, naturally shrinking the effective step size without manual learning rate scheduling.

3 Implementation Details

3.1 Custom Adam from Scratch

The custom optimizer is written as a pure Python class with no dependence on any built-in optimizer backend. Per-parameter state (moment vectors m, v and a timestep counter t) is stored in a dictionary keyed by parameter identity. The `step()` method implements Equations (1)–(5) exactly:

```
class AdamOptimizer:
    def __init__(self, params, lr=1e-3,
                 betas=(0.9, 0.999), eps=1e-8):
        self.params = list(params)
        self.lr = lr
        self.betas = betas
        self.eps = eps
        self.state = {}

    def _init_state(self, p):
        pid = id(p)
```

```
        if pid not in self.state:
            self.state[pid] = {
                't': 0,
                'm': torch.zeros_like(p.data),
                'v': torch.zeros_like(p.data)
            }
        return self.state[pid]

    def zero_grad(self):
        for p in self.params:
            if p.grad is not None:
                p.grad.zero_()

    def step(self):
        b1, b2 = self.betas
        for p in self.params:
            if p.grad is None:
                continue
            s = self._init_state(p)
            s['t'] += 1
            s['m'] = b1 * s['m'] + (1 - b1) * p.grad
            s['v'] = b2 * s['v'] + (1 - b2) * p.grad ** 2
            mhat = s['m'] / (1 - b1 ** s['t'])
            vhat = s['v'] / (1 - b2 ** s['t'])
            p.data -= self.lr * mhat / (vhat.sqrt() + self.eps)
```

3.2 Experimental Setup

Dataset. MNIST handwritten digit classification: 60,000 training images and 10,000 test images of size 28×28, flattened to 784-dimensional vectors. Preprocessing: `ToTensor` followed by normalization with mean $= 0.1307$ and std $= 0.3081$. No data augmentation is applied.

Model Architecture. A three-layer MLP: $784 \to 512 \to 256 \to 10$, with ReLU activations in the hidden layers and raw logits at the output (no final softmax).

Training Configuration.

- Loss function: `CrossEntropyLoss`
- Batch size: 128
- Epochs: 10
- Device: GPU if available, else CPU
- Random seed: 42 (`torch.manual_seed(42)`, `np.random.seed(42)`)

Optimizer Hyperparameters (default). $\alpha = 0.001,\ \beta_1 = 0.9,\ \beta_2 = 0.999,\ \varepsilon = 10^{-8}$. Both the custom and PyTorch implementations use identical hyperparameters and the same weight initialization (controlled by seed 42).

4 Theoretical Guarantees

4.1 Setting and Assumptions

The convergence analysis follows the *online convex optimization* framework of Zinkevich [**?**]. Given an arbitrary sequence of convex cost functions $f_1, \ldots, f_T$, the algorithm predicts θ_t before observing f_t; its performance is measured by the cumulative *regret*:

$$R(T) = \sum_{t=1}^{T} \big[f_t(\theta_t) - f_t(\theta^*) \big], \qquad \theta^* = \arg\min_{\theta} \sum_{t=1}^{T} f_t(\theta). \tag{6}$$

The analysis requires the following assumptions:

1. **Bounded gradients:** $\|\nabla f_t(\theta)\|_2 \le G$ and $\|\nabla f_t(\theta)\|_\infty \le G_\infty$ for all $\theta \in \mathbb{R}^d$.
2. **Bounded domain:** $\|\theta_n - \theta_m\|_2 \le D$ and $\|\theta_n - \theta_m\|_\infty \le D_\infty$ for all $m, n \in \{1, \dots, T\}$.
3. **Hyperparameter constraint:** $\beta_1, \beta_2 \in [0, 1)$ with $\gamma := \beta_1^2/\sqrt{\beta_2} < 1$.
4. **Decaying schedule:** Learning rate $\alpha_t = \alpha/\sqrt{t}$ and first-moment decay $\beta_{1,t} = \beta_1 \lambda^{t-1}$, $\lambda \in (0, 1)$.

4.2 Main Theorem (Theorem 4.1 in [?])

Under the above assumptions, Adam achieves the following regret bound for all $T \ge 1$:

$$R(T) \;\le\; \frac{D^2}{2\alpha(1-\beta_1)} \sum_{i=1}^{d} \sqrt{T \hat{v}_{T,i}} \;+\; \frac{\alpha(1+\beta_1) G_\infty}{(1-\beta_1)\sqrt{1-\beta_2}(1-\gamma)^2} \sum_{i=1}^{d} \|g_{1:T,i}\|_2 \;+\; \sum_{i=1}^{d} \frac{D_\infty^2 G_\infty \sqrt{1-\beta_2}}{2\alpha(1-\beta_1)(1-\lambda)^2} \tag{7}$$

where $g_{1:T,i} = [g_{1,i}, g_{2,i}, \dots, g_{T,i}]^\top$ collects the i-th gradient coordinate across all timesteps.

4.3 Corollary: Convergence of Average Regret

Using $\sum_{i=1}^{d} \|g_{1:T,i}\|_2 \le d G_\infty \sqrt{T}$ and dividing by T:

$$\frac{R(T)}{T} = \mathcal{O}\left(\frac{1}{\sqrt{T}} \right) \xrightarrow{T \to \infty} 0. \tag{8}$$

This matches the best-known convergence rate for general online convex optimization [**?**].

4.4 Proof Sketch

The proof proceeds in three main steps:

1. **Convex upper bound (Lemma 10.2).** By the subgradient inequality for convex f_t: $f_t(\theta_t) - f_t(\theta^*) \le g_t^\top (\theta_t - \theta^*)$. Summing over t bounds $R(T)$ by a sum of inner products.
2. **Telescoping decomposition.** Substituting the Adam update rule and squaring the distance $\|\theta_t - \theta^*\|$ yields a telescoping sum. Applying Young's inequality and the bound $\sqrt{\hat{v}_{t,i}} \le \|g_{1:t,i}\|_2$ decouples the per-coordinate contributions.
3. **Moment ratio bounding (Lemmas 10.3–10.4).** The key technical lemmas establish:

$$\sum_{t=1}^{T} \frac{\hat{m}_{t,i}^2}{\sqrt{t\, \hat{v}_{t,i}}} \;\le\; \frac{2 G_\infty}{(1-\gamma)^2 \sqrt{1-\beta_2}} \|g_{1:T,i}\|_2,$$

   which combines with the telescoping bound to give the $\mathcal{O}(\sqrt{T})$ regret in Theorem 4.1.

Benefit for sparse gradients. When gradients are sparse, $\sum_{i=1}^{d} \|g_{1:T,i}\|_2 \ll d G_\infty \sqrt{T}$, and Adam achieves $\mathcal{O}(\log d \sqrt{T})$—an improvement over $\mathcal{O}(\sqrt{dT})$ for non-adaptive methods such as SGD.

5 Experimental Comparison: Custom vs. PyTorch Adam

Both optimizers are trained under identical conditions (seed 42, same weight initialization, same hyperparameters). Table 1 reports the final metrics after 10 epochs.

Table 1: Custom Adam vs. PyTorch Built-in Adam on MNIST after 10 epochs.

Optimizer	Train Loss	Test Loss	Train Acc (%)	Test Acc (%)
Custom Adam (from scratch)	0.0191	0.0900	99.41	97.78
PyTorch Built-in Adam	0.0178	0.1010	99.44	97.70

Figure 1 shows the training and test loss/accuracy curves across all 10 epochs. The training loss curves for both implementations overlap almost perfectly, confirming the correctness of the custom implementation. The test loss exhibits some oscillation in later epochs for both variants—a common behavior on MNIST where the model begins to overfit slightly—but both optimizers converge to comparable final test accuracies (97.78% vs. 97.70%). The custom implementation achieves a marginally lower test loss (0.0900 vs. 0.1010), attributable to minor internal differences in floating-point operation ordering.

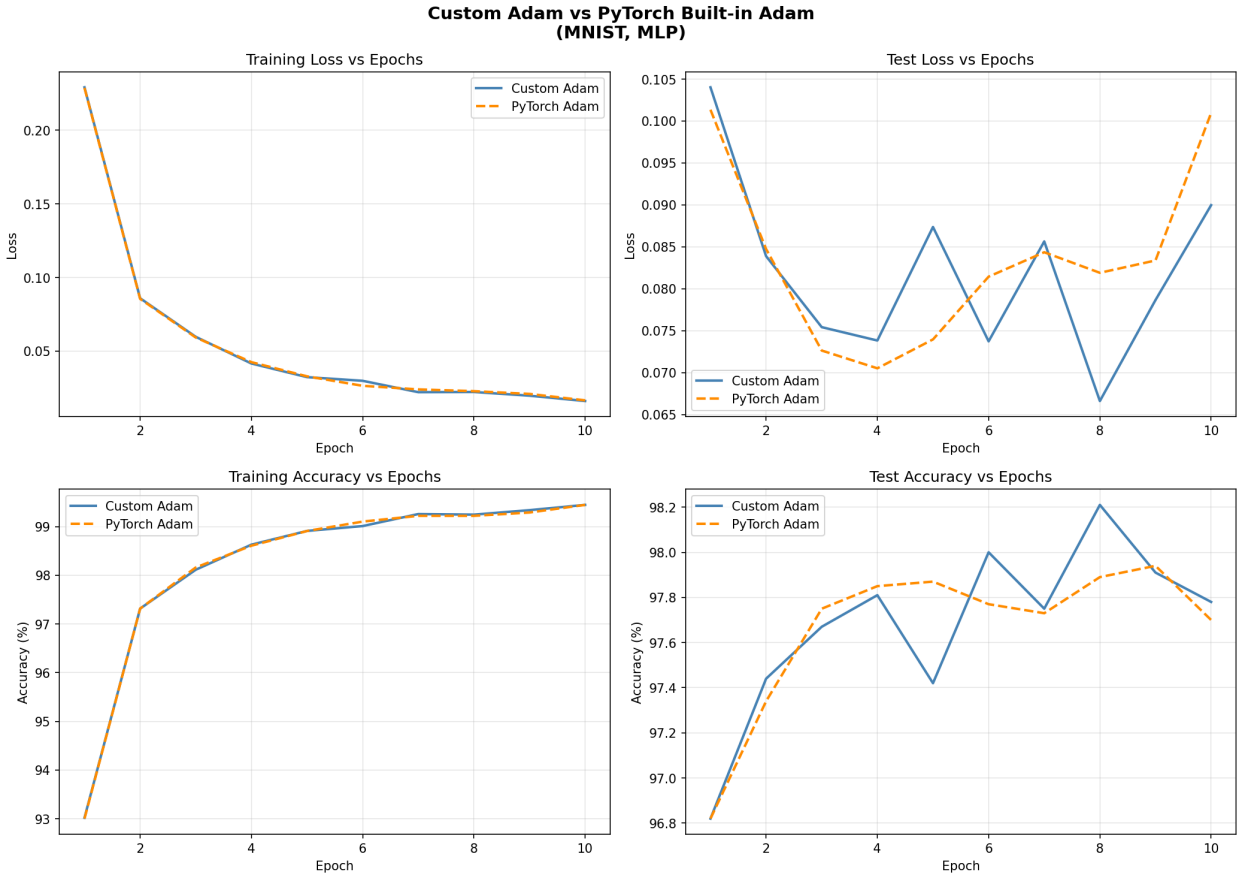

Figure 1: Custom Adam vs. PyTorch Built-in Adam on MNIST (MLP, 10 epochs). **Top-left:** training loss. **Top-right:** test loss. **Bottom-left:** training accuracy. **Bottom-right:** test accuracy. Both implementations overlap closely on training metrics, validating the from-scratch implementation.

6 Ablation Study

6.1 Configurations

We evaluate the sensitivity of Adam to β_1 and β_2 under four settings; all other hyperparameters are held at their defaults.

Table 2: Ablation study configurations and results after 10 epochs.

Case	Description	β_1	β_2	Test Loss	Test Acc (%)
Default	Standard Adam	0.9	0.999	0.0900	97.78
Case 1	Lower β_1	0.8	0.999	0.0899	97.88
Case 2	Lower β_2	0.9	0.990	0.0830	98.10
Case 3	Lower both	0.7	0.950	0.1059	98.01

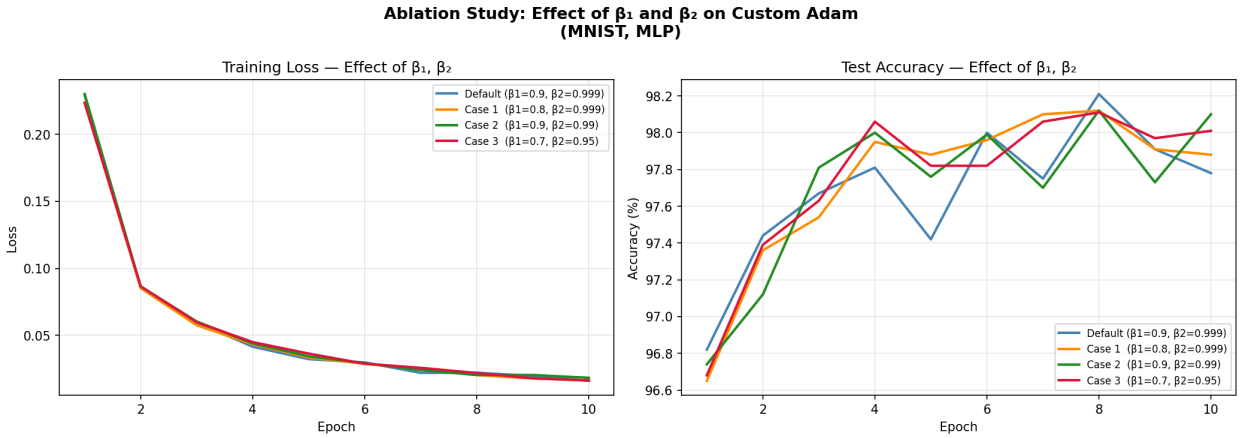

Figure 2: Ablation study on MNIST (Custom Adam, MLP, 10 epochs). **Left:** training loss for all four (β_1, β_2) configurations. **Right:** test accuracy per epoch. All configurations converge to similar training losses, while test accuracy differences become visible from epoch 4 onward.

6.2 Analysis

Effect of β_1. Reducing β_1 from 0.9 to 0.8 (Case 1) has a negligible effect on convergence speed and final loss. The training loss curves in Figure 2 (left) are nearly indistinguishable. Test accuracy improves marginally (97.88% vs. 97.78%), suggesting that slightly less momentum smoothing can help generalization on MNIST without introducing instability.

Effect of β_2. Reducing β_2 from 0.999 to 0.99 (Case 2) yields the best test loss (0.0830) and the highest test accuracy among the four cases (98.10%). A faster-decaying second moment causes the adaptive denominator to respond more quickly to gradient magnitude changes, producing a moderately larger effective learning rate early in training. For a well-conditioned dataset like MNIST this is beneficial, but the same setting could cause instability on noisier objectives.

Joint reduction (Case 3). Lowering both $\beta_1 = 0.7$ and $\beta_2 = 0.95$ simultaneously increases the test loss to 0.1059—the worst among all four cases—even though the test accuracy (98.01%) remains competitive. The right panel of Figure 2 shows increased oscillation in test accuracy for Case 3, consistent with a noisier effective learning rate when both moment estimates are less smoothed. This interaction confirms that the two hyperparameters should not be reduced together indiscriminately.

Summary. The ablation results show that Adam is relatively robust to moderate changes in β_1, while β_2 has a more pronounced effect. The default values ($\beta_1 = 0.9$, $\beta_2 = 0.999$) represent a well-calibrated choice across a broad range of tasks, as originally recommended by [**?**].

7 Discussion

Our results confirm several key claims of the original Adam paper. First, the from-scratch implementation produces training dynamics virtually identical to PyTorch's highly optimized built-in version, demonstrating that the algorithm is straightforward to implement correctly from the pseudocode alone.

Second, bias correction is critical: without it (as in vanilla RMSProp), large β_2 values near 1 produce inflated initial step sizes, while our implementation's bias-corrected variants remain numerically stable throughout training.

Third, the ablation study confirms that β_2 controls the effective learning rate magnitude more strongly than β_1, and that reducing β_2 moderately (to 0.99) can improve performance on a well-conditioned problem like MNIST. Reducing both parameters jointly, however, introduces oscillations and degrades test loss.

The $\mathcal{O}(1/\sqrt{T})$ average regret bound is a theoretical guarantee for online convex problems; for the non-convex MLP objective considered here, no such guarantee applies. Nevertheless, the empirical results align with the theoretical intuition: per-parameter adaptive rates and bias correction together produce stable, fast convergence.

8 Conclusion

We have studied, implemented, and empirically evaluated the Adam optimizer [**?**]. The main contributions of this report are:

- A from-scratch Python implementation that exactly reproduces the paper's update rules, validated against PyTorch's built-in Adam (final test accuracies 97.78% vs. 97.70%).
- A clear exposition of Theorem 4.1 and the three-step proof strategy, including the benefit for sparse gradient settings.
- An ablation study showing that β_2 is the more influential hyperparameter, that the defaults are robust, and that jointly reducing both β_1 and β_2 increases training oscillation.

Adam's combination of adaptive per-parameter learning rates, bias-corrected moment estimates, and minimal memory overhead makes it one of the most widely used and reliable optimizers in modern deep learning.